Head Start or Process?
Decomposing How an Experience-Acquired Abstraction Shapes Later Learning in a Small Language Model

Sam Ramdan
Independent Researcher
sr4729820@gmail.com

Code and data: https://github.com/Somebodyhere101/head-start-or-process

Prepared with substantial AI assistance (Claude, Anthropic); see Statements.

October 2026

Abstract

When a network that has acquired an abstraction A learns a second abstraction B faster, does A supply information or change the *process* by which B is learned? We decompose a gradient learner's function-space update into starting predictions, evidence and tangent kernel, and isolate the kernel channel with an *output-matched* learner: A's network with a fixed stop-gradient offset, so that it starts B with the naive model's predictions. We apply it to rank-8 adapters on Qwen3-1.7B that learned a hidden two-cue conjunction from outcome-labelled episodes. The compact layer-24 mediator of A's transfer is the answer-letter readout (one adapter tested fully, five on the training domains with fewer candidates). A's raw advantage is a head start plus a network effect that operates only from A's own saturated predictions; from matched starts, A's network gives no detectable mean speed-up. Near-inert replicates of the base learner differ by up to 0.28 in mean held-out balanced accuracy, so single-run comparisons, including our preregistered ones, are dominated by trajectory noise. Against replicated references (post hoc), the mean kernel-channel benefit is below $+0.02$ at the onset of learning on a task whose rule A does not share, and below $+0.09$ on a task that extends A, where the scaffold question stays open. Two post hoc observations suggest the method is not blind: all three shuffled-outcome adapters slowed matched-start learning (not significant at the adapter level), and one of four kernel statistics ranked the six adapters' matched-start advantage (Spearman 0.94 on both tasks, uncorrected). The preregistered positive control failed calibration, so the frozen verdict is uninterpretable.

1 Introduction

Prior learning makes later learning easier, in animals (Harlow, 1949; Tse et al., 2007) and in networks (Raghu et al., 2020; Goudar et al., 2023). Whether it does so by supplying *information* or by changing the *process* of learning is an old and still open question. Raghu et al. asked it of MAML ("rapid learning or feature reuse?") and found feature reuse (Raghu et al., 2020). In recurrent networks trained without a meta-objective, a schema subspace speeds later learning (Goudar et al., 2023). In language models, earlier fine-tuning can change how later updates store facts (Krasheninnikov et al., 2024).

This paper asks the question of a specific, behaviourally characterised, experience-acquired abstraction in a language model. Three requirements make the question sharp.

Learning curves cannot answer it. A learner whose parameters change from θ_0 to θ_A is a fixed update rule with more knowledge, and also a changed update rule; the two descriptions are equivalent (cf. Finn and Levine, 2018). "A changed the learner" is therefore trivially true. The substantive claim must be about *which channel* carries A's influence on B.

Channels. For a gradient learner with a fresh adapter ϕ on top of θ, the first-order function-space update on input x after step t is

$$f_{t+1}(x) = f_t(x) - \eta\, K_t(x, x_t)\, \nabla_f \ell\big(f_t(x_t), y_t\big), \qquad K_t = J_t P_t J_t^\top, \tag{1}$$

where J_t is the Jacobian with respect to ϕ and P_t the optimizer's preconditioner (Jacot et al., 2018; Malladi et al., 2023). A can act through (i) the starting predictions f_0, which also set the residuals and, in our update, the policy weights; (ii) the evidence stream (x_t, y_t); or (iii) the kernel K, i.e. how evidence about one item moves predictions on others. We call (i) and (ii) information and (iii) process. Reusable features of A enter through J, so they belong to (iii) unless they also shift f_0.

Identification and removal. A strong process claim would identify A inside the network, show that it acts through the kernel and not through predictions, and show that B, once formed with A's help, stands without A. That last condition separates a *scaffold* for forming B from a *component* of B.

Contributions.

1. A decomposition built from three tools. An output-matched learner isolates the kernel channel; a prior-shift learner holds A's starting predictions on base's network; and abrupt and gradual removal tests separate reuse from scaffolding. All of it works in full nonlinear training, extending the linearised kernel swaps of Ortiz-Jiménez et al. (2021).
2. A localisation result. The compact mediator of a consequence-learned skill is the answer-letter readout, with no single-site, format-free address at the sites tested. This corrects the earlier reading of our own mediation results.
3. A bounded mean null. From matched starting predictions, the acquired abstraction gives no detectable mean speed-up of later learning, although it changes the tangent kernel by orders of magnitude. Two post hoc observations suggest the method is not blind to process effects: saturated rule-free adapters slowed matched-start learning, and one kernel statistic ranked the adapters' matched-start advantage. Both are hypotheses; we froze the second as a forecast for four held-out adapters.
4. A noise result with a correction. In this regime learning is jump-like, and near-inert perturbations move single-run outcomes by up to 0.28. Our preregistered design used one reference run per seed; replicating the reference changes the sign of two contrasts and shrinks a preregistered scaffold collapse from -0.27 to -0.10, leaving the scaffold question open on one task. We report the preregistered and the corrected analyses side by side.

2 Related Work

Learning to learn and feature reuse. Learning sets (Harlow, 1949) and schema-dependent consolidation (Tse et al., 2007; Maor et al., 2026) are the behavioural precedents. MAML-style analyses separate rapid learning from feature reuse (Raghu et al., 2020). Arnold et al. (2021) show that collapsing learned linear layers preserves initial predictions but destroys fast adaptation; this is the closest analogue of our output-matched null. Goudar et al. (2023) identify a schema subspace whose perturbation abolishes learning-to-learn in RNNs. Krasheninnikov et al. (2024) show implicit meta-learning in LMs: an earlier stage changes how later updates internalise facts. None of these separates a specific experience-acquired A's kernel effect from its head start under matched information, or tests formation against use.

Interventions during training. Ablating concept directions during fine-tuning only changes what is learned (Casademunt et al., 2025). Adding persona directions during fine-tuning and removing them at test prevents trait acquisition (Chen et al., 2025), and that protection depends on ongoing adaptation, not on a static weight offset (Guan et al., 2026). Suppressing an internal structure early changes later acquisition, with a closing window (Chen et al., 2024). A transplanted circuit lets a later strategy form (Singh et al., 2025). Abrupt removal of a training-time prior destroys what gradual removal preserves (Yang et al., 2026). Early-learning subspaces shape later representation change (Deng et al., 2026). Frozen logit offsets added during training and removed at test are an established device in debiasing (Clark et al., 2019). We use the device in a new role.

Kernels and transfer. Fine-tuning of prompted LMs is often kernel-like (Malladi et al., 2023). Linearised models with a trained network's kernel and zero initial function train faster but generalise worse (Ortiz-Jiménez et al., 2021), and label–gradient alignment predicts fine-tuning (Deshpande et al., 2021). Ren and Sutherland (2025) decompose the per-step influence of LLM fine-tuning updates in the same function-space form and show a "squeezing" effect on peaked outputs, a relative of the saturation effects we find. A model's starting predictions also shape which features later fine-tuning changes (Kumar et al., 2022). Softmax policy gradient slows sharply near saturated policies (Mei et al., 2020). Fine-tuning tends to enhance existing mechanisms (Prakash et al., 2024). Kernel similarity is commonly measured by CKA (Kornblith et al., 2019).

Plasticity and readouts. Prior training can also reduce later trainability: warm-started networks generalise worse (Ash and Adams, 2020), and networks lose plasticity in continual learning (Lyle et al., 2023; Dohare et al., 2024). Our question is the converse, whether prior learning improves the learning process. Multiple-choice answers in LMs are produced by dedicated letter-selection mechanisms (Lieberum et al., 2023; Wiegreffe et al., 2025), which is where our Stage 0 finds A's compact mediator. Interchange interventions follow Geiger et al. (2024); subspace patching can mislead (Makelov et al., 2024).

Variance in fine-tuning. Fine-tuning outcomes vary strongly with random seed, data order and initialisation, and some runs fail outright (Dodge et al., 2020; Mosbach et al., 2021). Benchmark comparisons that ignore this variance mislead (Bouthillier et al., 2021). Crossed random factors call for resampling both factors (Owen, 2007), and few clusters make cluster bootstraps anti-conservative (Cameron et al., 2008). Our Section 6.2 shows the same problem for mechanistic training contrasts.

Our prior work. The A adapters come from our earlier, unpublished consequence-learning study. It reported far transfer and four mediating directions at layer 24; Section 5 reinterprets those directions.

3 Setup

World and abstraction A. Situations describe three anomalies logged at a site. The hidden rule is: one shared cause iff the anomalies began in the same time window *and* their carriers share an infrastructure. The model answers a four-option letter question ("What is the best explanation?"; option order is shuffled). A is a rank-8 LoRA on the query and value projections of every layer of Qwen3-1.7B. It is trained with an exact policy-gradient update over the four letters from revealed outcomes, on outcome-balanced plant and clinic situations (3 passes over 384 situations). It reaches 0.955–1.000 on held-out software situations on 10/10 seeds. Three caveats: the update reveals every letter's reward, so it is outcome-label supervision weighted by the model's own policy rather than bandit learning from its own choices; the domains share one sentence template; and about a third of the transfer is lexical. Six of the ten adapters, from three training series, were designated for this study in the Stage-1 preregistration; the other four are held out for confirmation.

Abstraction B. A fresh rank-8 adapter ϕ is trained on the software domain, on the interpret question only, with the same update.

B_ind One shared cause iff same reporting desk AND same work order; time and infrastructure are drawn independently, so A's rule is irrelevant. (Its cues are repeated strings, like A's infrastructure cue, so A still starts it above chance.)

B_ext Time AND infrastructure AND desk; this extends A.

B_para Same batch value AND same supplier city, each written in three different surface forms per item (numeral, word, arithmetic; name, nickname, description). Sameness is semantic and never a repeated string (preregistered for Stage 3; not run, Section 7).

Test sets are half positives and half near-misses (one cue false). We report *balanced accuracy*: the mean of positive accuracy and near-miss accuracy. It is 0.5 for a constant policy, at most 0.75 for any single-cue policy, and 1.0 for the conjunction.

Update. We use the frozen recipe's exact update (temperature 2, $\epsilon = 0.1$ mixture on the policy weights, moving baseline, Adam 5×10^{-4}, batch 4; full settings in Appendix B). One change was made before any Stage-2 result was seen (after a smoke test): the KL anchor is a proper four-letter KL, because the original unnormalised form overflows under output offsets.

4 The Decomposition

Let $\ell p_\theta(x) \in \mathbb{R}^4$ be the letter log-probabilities, and let $\delta(x) = \ell p_{\theta_A}(x) - \ell p_{\theta_0}(x)$, computed once per prompt and frozen. Every arm uses the same items, item order, ϕ initialisation and update.

base Network θ_0; no offset.

A_raw Network θ_A; no offset. This is the transfer effect as usually measured.

prior Network θ_0; offset $+\,\mathrm{sg}[\delta]$. This holds A's starting predictions on base's network. It was preregistered as the information-matched null.

A_matched Network θ_A; offset $-\,\mathrm{sg}[\delta]$. At $t = 0$ its predictions, policy weights and KL anchor equal base's exactly (maximum deviation $\leq 8 \times 10^{-6}$), while $\partial f / \partial \phi$ is A's. Two terms are not matched. The letter-mass term (weight 0.1) is computed on the network's raw output (D1), so it is A's. And gradient-norm clipping binds less often for A's network (Section 6.3).

A_matched_fade As A_matched, with ΔA scaled from 1 to 0 over the last third of the budget. Offsets were tabulated on a 0.1 grid of the scale and applied at the nearest grid point, while the weights used the exact scale, so matching during the fade is approximate (an implementation flaw found in review).

Controls Adapters trained with the same recipe on shuffled outcomes (*shuf*: saturated, no rule), on the infrastructure-only rule (*single*), or A's factors permuted (*perm*: norm-matched, near-inert). Each is run output-matched, and in an exploratory follow-up (E2) shuf and single were also run raw and prior.

Let G be the mean balanced held-out accuracy at eight evenly spaced points over the budget. The contrasts are:

- $\Delta_{\text{raw}} = G(\text{A_raw}) - G(\text{base})$;
- $\Delta_{\text{start}} = G(\text{prior}) - G(\text{base})$;
- $\Delta_K = G(\text{A_matched}) - G(\text{base})$;

- S, the accuracy of $\theta_0 + \phi$ (A unmerged) learned under A_matched, minus that of base's own ϕ; and S_{fade}, the analogous contrast for the fade arm;
- the interaction $(\Delta_{\text{raw}} - \Delta_{\text{start}}) - \Delta_K$. Its first part, $G(\text{A_raw}) - G(\text{prior})$, compares A's and base's networks from the same starting predictions (namely A's); we call it the *start-conditional network effect*.

H2 (process) predicts $\Delta_K > 0$, specific to A relative to the controls. The scaffold form additionally predicts $S > 0$.

5 Stage 0: Where Is A?

We searched for a letter-agnostic representation of A's decision. Each situation was presented in all four cyclic option rotations, so that letter identity is decorrelated from meaning. We patched candidate subspaces between items of opposite decision and different rotation, and scored whether the patched choice follows the donor's *meaning* or the donor's *letter*. Both are measured as excess over the unpatched pairs.

Sites were the final position (layers 16–26) and the last token of each decision option's text (layers 8–24). Candidates were the top PCs of A's deltas, letter-residualised PCs, a class-mean decision direction, random and base-letter controls, and the whole residual.

Result (6/6 adapters; adapters 2–6 on the fit domains only, Appendix F).

- No tested site or layer transfers meaning: excess meaning-flip is 0.00 everywhere, including whole-residual swaps at both option tokens and at the final position through layer 20. Cue tokens inside the situation text were not tested.
- At layers 24–26, the final position carries the answer letter. A 4-dimensional subspace reproduces the whole-residual swap exactly: excess letter-copy 0.64, meaning-flip 0.00.
- For adapter 1, 61% (layer 24) and 73% (layer 26) of the A/B/C/D unembedding span lies inside that subspace, against 0.1% for random 4-dimensional subspaces.

The "four mediating directions" of our earlier work are therefore the letter readout. At the sites tested, A's decision has no single-site, format-free address; it may be distributed across positions, which single-site patching cannot test. Per the preregistration, the localisation test (removing A's representation during B-learning only) was dropped.

6 Results at 1.7B (Stage 2)

The design is 6 A adapters $\times$ 3 B seeds $\times$ 2 tasks, plus 9 control adapters: 204 preregistered B-learning runs, plus 72 exploratory runs (E2). Adapters and B seeds are crossed: every adapter learns the same three B item sets, and each seed has a single base run that serves as the reference for every contrast. Budgets were set by a frozen pilot rule (the smallest number of passes at which base reached balanced accuracy ≥ 0.85 on one or two of three pilot seeds): one pass (192 situations) for B_ind and three passes for B_ext.

6.1 Preregistered analysis

As frozen, Δ_K was equivalent to zero on B_ind (90% CI within ± 0.05), A_matched fell behind base late in B_ext, and B did not survive abrupt removal of A on B_ext ($S = -0.27$). No primary contrast favours H2 (Holm-adjusted one-sided $p \geq 0.75$ for all four on both tasks). The frozen verdict is nevertheless

Contrast	B_ind (A's rule irrelevant)	B_ext (A extends)
Δ_{raw}	$+0.156$ [0.098, 0.211]	$+0.283$ [0.204, 0.348]
Δ_{start}	$+0.067$ [0.036, 0.099]	$+0.169$ [0.135, 0.203]
Δ_K	-0.014 [-0.054, 0.024]	-0.029 [-0.098, 0.047]
S (abrupt unmerge)	-0.035 [-0.070, 0.004]	-0.273 [-0.347, -0.202]
S_{fade}	$+0.056$ [-0.020, 0.145]	-0.048 [-0.123, 0.019]
$\Delta_K - \Delta_K(\text{shuf})^\dagger$	$+0.004$ [-0.064, 0.091]	$+0.063$ [-0.079, 0.188]
$\Delta_K - \Delta_K(\text{single})^\dagger$	-0.101 [-0.221, 0.027]	$+0.032$ [-0.049, 0.096]
$\Delta_K - \Delta_K(\text{perm})^\dagger$	-0.043 [-0.099, 0.023]	$+0.030$ [-0.106, 0.183]
Exploratory, post hoc (E1, E2)		
Interaction, A adapters	$+0.103$ [0.048, 0.170]	$+0.143$ [0.083, 0.216]
Interaction, shuf controls	$+0.069$ [-0.010, 0.151]	$+0.129$ [0.053, 0.212]
Interaction, single controls	-0.043 [-0.152, 0.052]	$+0.181$ [0.067, 0.309]
Learning rate $\times 2$ (pure process)	$+0.074$	-0.109

Table 1: Preregistered analysis, Qwen3-1.7B. Differences in G (balanced accuracy) against the single base run of each B seed; 95% hierarchical-bootstrap CIs that resample adapters, then B seeds within adapter. Section 6.2 shows why these intervals are too narrow and why the base reference is biased on B_ext. †Paired with the control trained from the same seed, so only the three A adapters that have one enter ($\Delta_K = -0.032$ on B_ind and -0.036 on B_ext for these three).

"uninterpretable", because the process-only positive control failed calibration (Section 6.4). Two features of the data, which the preregistration did not anticipate, change how Table 1 should be read.

6.2 How noisy is one training run?

The three perm controls are A's LoRA factors, permuted. Their letter KL to base is ≤ 0.01 (mean absolute letter log-probability difference 0.6–1.5 nats, mostly on improbable letters, and cancelled at $t = 0$ by the offset), and their tangent kernel on B matches base's (Appendix G). Run output-matched, they are the base learner with a near-inert perturbation of the weights, with ϕ's initialisation and the item order unchanged. They should reproduce base's learning curve. They do not (Figure 1, Appendix D). On B_ext seed 101 the four runs reach $G = 0.42$–0.71 and final accuracy 0.42–0.99, and one run reaches 1.00 and then collapses to 0.50. The within-seed SD of G is 0.077 on B_ext and 0.027 on B_ind. On B_ext seed 101, the base run, the reference for every preregistered contrast, is the best of the four, $+0.127$ above their mean.

The noise comes from jump-like learning. In the pilot, base's B_ind accuracy jumped from ≤ 0.65 to ≥ 0.92 within half a pass, at between half a pass and one and a half passes depending on the seed. On B_ext the jump came at three passes, at six passes, or not within six passes. G therefore mostly records when the jump happens, and small perturbations move that time. The frozen budget rule placed each budget where base had jumped on only one or two of three pilot seeds, which is where this variance is greatest. On B_ind the budget ends at the onset of learning: the reference runs reach $G = 0.50$–0.62 and final accuracy 0.51–0.75. B_ind contrasts therefore concern only the onset of learning.

6.3 Robustness re-analysis (post hoc)

We re-estimated every contrast against a four-run reference per seed: base and the three output-matched perm runs (E3, post hoc, logged before writing this section). Intervals come from a three-way bootstrap that resamples adapters, B seeds and reference runs within seed. As checks, Table 2 also gives t intervals over the six adapter means, which ignore seed and reference noise, and a stratified permutation test of output-matched runs against reference runs within seed.

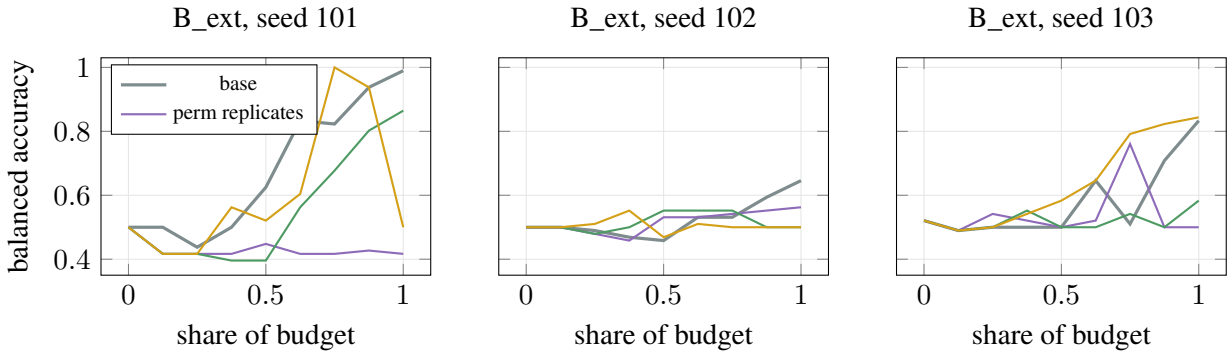


Figure 1: Four runs of the base learner on identical B_ext data: base itself and three output-matched perm controls, whose weights differ from base's by a near-inert perturbation (letter KL ≤ 0.01). Learning is jump-like, and the timing of the jump, and whether it holds, varies from run to run.

No detectable process benefit from a matched start. Against the reference, Δ_K is -0.022 on B_ind and $+0.021$ on B_ext, and neither differs from the reference runs (permutation $p = 0.22$ and 0.59). Two-sided equivalence within ± 0.05, as preregistered, holds on neither task. The H2 direction, however, is bounded. The one-sided 95% upper bound is $+0.02$ on B_ind and $+0.09$ on B_ext (bootstrap and t agree to 0.003). On B_ind this excludes a process effect the size of doubling the learning rate (Section 6.4), but only at the onset of learning. On B_ext it excludes effects of our preregistered smallest effect size of interest (0.10), barely. The late lag of A_matched behind base on B_ext was the lucky base run: A_matched lies inside the reference band (Figure 2).

Can the output-matched learner register kernel-channel effects? Pooled over runs, rule-free saturated adapters slow matched-start learning on B_ind (-0.044 [-0.088, -0.008]; run-level permutation $p = 0.005$). Those nine runs come from three adapters, and neither the permutation test nor a bootstrap over three adapters handles that clustering. At the adapter level all three are negative (-0.015, -0.081, -0.038), but the 95% t interval is [-0.127, $+0.039$]. On B_ext the adapter means are -0.065, -0.086 and $+0.002$. Their starting predictions are matched to base's, so any such effect is a kernel-channel effect, and a negative one. This suggests, but does not establish, that the method can detect process changes; the preregistered positive control could not be calibrated. On B_ind, A lies between shuf and the reference. A rule-specific benefit of A could thus be partly masked by the same saturation penalty. The paired contrasts against shuf in Table 1 are the relevant test, and with three pairs they are too wide to decide it.

Completion and stability. G hides two differences between A_matched and the reference. First, completion: within budget, no A_matched run reached balanced accuracy 0.85 on either task (0/18 each), against 3/12 reference runs on B_ext (one-sided Fisher $p = 0.05$) and 0/12 on B_ind. The preregistered labels-to-criterion outcome is therefore censored for nearly every run, and we report these rates instead. Second, stability: 21% of A_matched's evaluation points on B_ind fall below 0.45 (minimum 0.31), against none for base or the perm replicates; on B_ext the rates are 23% and 19%. Below-chance accuracy means ϕ has pushed the output against the rule. A plausible route is A's high-gain readout: a small move of ϕ along it re-injects A's decision, with either sign, into the matched frame. Gradient-norm clipping also binds at 51–79% of A_matched's logged evaluation points, against 99–100% for base and perm. These are process differences, and they do not speed learning.

Contrast vs. reference	Estimate	95% CI, three-way bootstrap	90% CI, t (df 5)	permutation p
B_ind (reference SD of G within seed 0.027)				
Δ_K (A)	-0.022	$[-0.067, +0.026]$	$[-0.065, +0.020]$	0.220
Δ_K (shuf)	-0.044	$[-0.088, -0.008]$	–	0.005
Δ_K (single)	$+0.060$	$[-0.133, +0.285]$	–	0.253
Δ_{raw}	$+0.147$	$[+0.087, +0.206]$	$[+0.097, +0.198]$	–
Δ_{start}	$+0.059$	$[-0.018, +0.122]$	$[+0.040, +0.077]$	–
Interaction	$+0.111$	$[+0.046, +0.207]$	$[+0.054, +0.168]$	–
S (abrupt unmerge)	-0.065	$[-0.135, -0.004]$	–	–
S_{fade}	$+0.027$	$[-0.081, +0.147]$	–	–
with A − without A	$+0.002$	$[-0.105, +0.109]$	–	–
B_ext (reference SD of G within seed 0.077)				
Δ_K (A)	$+0.021$	$[-0.066, +0.106]$	$[-0.049, +0.090]$	0.589
Δ_K (shuf)	-0.050	$[-0.136, +0.024]$	–	0.140
Δ_K (single)	-0.019	$[-0.102, +0.068]$	–	0.565
Δ_{raw}	$+0.333$	$[+0.222, +0.419]$	$[+0.267, +0.399]$	–
Δ_{start}	$+0.219$	$[+0.147, +0.285]$	$[+0.203, +0.235]$	–
Interaction	$+0.094$	$[+0.013, +0.179]$	$[+0.031, +0.156]$	–
S (abrupt unmerge)	-0.095	$[-0.259, +0.033]$	–	–
S_{fade}	$+0.130$	$[-0.013, +0.260]$	–	–
with A − without A	$+0.118$	$[-0.017, +0.212]$	–	–

Table 2: Post hoc re-analysis (E3) against the four-run reference. Δ_K rows: output-matched runs of A and of the rule-free (shuf) and single-cue controls. Scaffold rows use the reference runs' own scaffold evaluations; "with A − without A" is A_matched's final accuracy minus that of the same ϕ with A unmerged (no reference needed). Permutation p is two-sided and treats runs within a seed as exchangeable. It ignores clustering by adapter, so it is anti-conservative for the control rows, which come from three adapters each.

The raw advantage: a head start and a start-conditional network effect. Against the reference, A's raw advantage is $+0.147$ (B_ind) and $+0.333$ (B_ext), and its frozen head start (Δ_{start}) is $+0.059$ and $+0.219$. The prior-shift learner cannot move from A's starting predictions (Figure 2). A's letter log-probabilities differ from base's by 75–82 nats on average, beyond what base's adapter can overturn within the budget. So Δ_{start} measures A's start held fixed, not what base's network could learn from A's information; in this regime the prior arm is a degenerate null. A's own network does revise those predictions. From the same start, A_raw exceeds prior by $+0.089$ [0.009, 0.196] on B_ind and $+0.114$ [0.042, 0.175] on B_ext. A plausible but untested mechanism is A's high-gain letter readout, whose kernel scale is up to 3,500× base's: it gives A's network the function-space step sizes needed to move saturated logits. From base's start, the same network gives no advantage.

Is the start-conditional effect specific to A? Unresolved. Our pre-written E2 rule classes the interaction as generic saturation, because shuffled-outcome adapters, which are almost as saturated as A (61–75 nats against 75–82), reach 67% and 90% of A's interaction. The rule was written after E1 had shown A's interaction. The single-cue controls, equally saturated (74–77 nats), contradict it on B_ind (interaction -0.043; Table 1). The components show that this reading does not hold. The controls reach that total by a different route: mostly through their matched-start slow-down ($-\Delta_K$), not through the start-conditional effect, where shuf reaches only $+0.034$ and $+0.030$. A minus shuf on that component is $+0.055$ $[-0.07, +0.17]$ (B_ind) and $+0.084$ $[-0.02, +0.18]$ (B_ext). The single-cue control matches A on B_ext ($+0.113$), where its cue is part of the rule, but not on B_ind ($+0.026$). We therefore withdraw the "generic saturation" reading: the E2 statistic did not isolate the component it was meant to test.

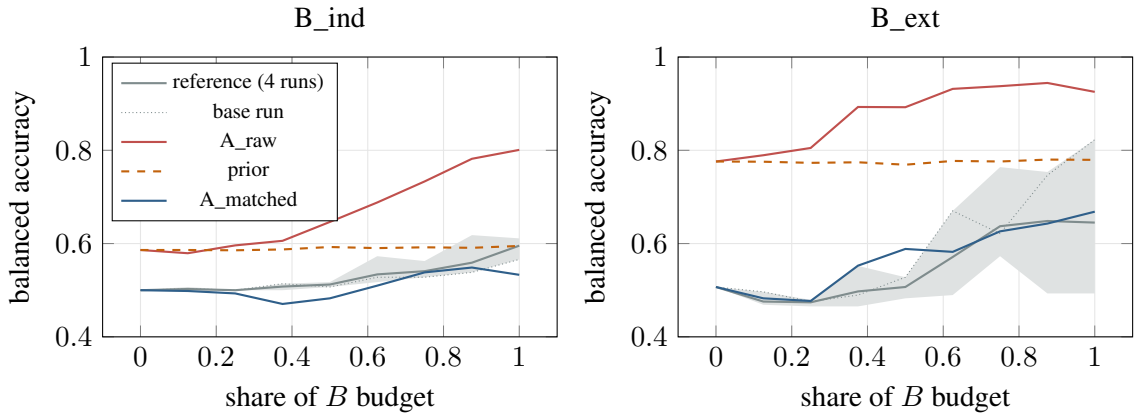


Figure 2: Mean learning curves at 1.7B (6 adapters $\times$ 3 B seeds). Grey band: range of the four reference configurations (base and three perm replicates), each averaged over seeds. The prior-shift learner (base network with A's saturated starting predictions) cannot move. A's network learns from its own predictions (A_raw), but from base's predictions (A_matched) it stays within the reference band.

Scaffold: none at the onset of learning on B_ind, unresolved on B_ext. On B_ind, where learning had barely begun (reference G 0.50–0.62), the B learned on top of A is slightly worse than the reference once A is removed ($S = -0.065$ [-0.135, -0.004]), and gradual removal does not help. On B_ext, the preregistered collapse ($S = -0.27$) was measured against the lucky base run. Against the reference, abrupt removal costs -0.095 [-0.259, $+0.033$]. A_matched's own B loses 0.118 [-0.017, 0.212] when A is unmerged, so B partly uses A. Gradual removal leaves a B that is, if anything, better than the reference ($S_{\text{fade}} = +0.130$ [-0.013, $+0.260$]), but the fade arm's offsets were only approximately matched (Section 4). Neither "no scaffold" nor "scaffold" is established on B_ext.

The kernel changes, but not usefully on average. Before any B update, A's tangent kernel on B prompts is 10–3,500$\times$ larger in scale than base's in 11 of 12 adapter–task pairs (one is 9$\times$ smaller). Its alignment with B's residuals is about 10$\times$ lower for the plain gradient kernel $JJ^\top$ (F_{res} 1.2–11.6 versus 75–85). It is 1.5–5$\times$ lower for the sign-gradient kernel, which approximates Adam's update (F_{sign} 18–58 versus 83–90), and negative for one adapter. Alignment with item types (CKA) is 0.01–0.08 against 0.46–0.53. Permuted controls match base closely. The preregistered forecast failed: across arm $\times$ cell at seed 101, no statistic reaches Spearman 0.5 with G (largest 0.23). Kernel statistics were measured on one B seed and are plain means, so they are sensitive to outliers.

Adapter-level kernel effects (post hoc). The forecast asked whether the kernel ranks arms. Asked instead whether it ranks adapters (E4, prompted by review), the answer differs. Across the six A adapters, F_{sign} of the matched frame ranks mean Δ_K at Spearman 0.94 on both tasks (exact two-sided $p = 0.017$ each, uncorrected; 0.067 after Bonferroni correction for the four statistics tried), and across all 15 networks at 0.60 ($p \approx 0.02$). F_{res}, CKA and kernel scale do not ($|\rho| \leq 0.71$, $p \geq 0.14$). The adapter with negative F_{sign} (ST2_20260922) has the worst Δ_K on both tasks (-0.089, -0.120). The shared base run cancels in a ranking across adapters, so trajectory noise of the reference cannot produce this. F_{sign} does not predict the level relative to base: base's F_{sign} exceeds every A adapter's, yet several adapters match base. With six adapters, four statistics and a post hoc question, this is a hypothesis. It says that the mean null averages over adapter-specific kernel effects of both signs. We have frozen it as a forecast for the four held-out adapters, before any B-learning run on them (Appendix I).

6.4 Sensitivity

The preregistered process-only control (amplifying updates in a calibration-gradient subspace) failed calibration. Its gains over base were $-0.06/-0.03/-0.05$ (B_ind) and $+0.07/+0.08/+0.01$ (B_ext) for factors 2/4/8. The frozen verdict is therefore "uninterpretable". A learning-rate benchmark added afterwards (E1) shows that G registers known process changes on B_ind: doubling the learning rate gives $+0.074$ (positive on all three seeds, about three times the standard error implied by the B_ind reference noise). On B_ext the same change gives -0.109, only 1.7 standard errors from zero given the B_ext reference noise. The benchmark itself used one run per seed. Stage 3 preregisters it.

7 Preregistered but Not Run, and What Would Settle It

A third stage was frozen (sha256 `89b30e9f...`) to close the gaps of Stage 2. It specified:

- a replication of the full design in Qwen3-0.6B, using three existing 0.6B A adapters, which transfer at 0.997–1.000;
- B_para at 1.7B and 0.6B, a task whose cue identity is semantic rather than a repeated string (Section 3);
- the learning-rate benchmark as the preregistered sensitivity control;
- the saturation test, now confirmatory.

The author stopped this stage during its pilot, after one pilot cell, to write this report. No Stage-3 result is analysed or reported here. The preregistration and generator are released. In light of this paper's problems, Stage 3 should be amended before it runs. A design that could settle the question needs:

1. at least ten B seeds, with replicated base references (re-initialised or perturbed) per seed;
2. budgets that take base past the learning jump on every seed, with time to criterion as the primary outcome;
3. the four held-out adapters as a confirmation set for the F_{sign} forecast, which is already frozen (Appendix I);
4. a learnable information-matched null, such as A's decision at base's confidence (temperature-scaled δ);
5. a matched learner with the mass term on the matched output, fp32 offsets, exact fade offsets, and a learning-rate sweep per learner;
6. the 0.6B replication and B_para.

On the hardware used here, this is one to two GPU-days.

8 Discussion

At this scale, an abstraction acquired from consequences shapes the learning of the next one mainly through its starting predictions, and through a network effect that operates only from those predictions. From a matched start we detect no mean benefit and bound it. Adapter-level kernel differences may exist and average out. Four lessons apply beyond our setting.

First, "A changed the learning process" must be operationalised against an information-matched null. Without the prior-shift and output-matched learners, our raw transfer of $+0.15$ to $+0.33$ would look like learning-to-learn. With them, it decomposes into a head start and an effect conditional on that start. The prior-shift null itself needs care: with saturated starts it measures a frozen start, not information.

Second, replicate the reference. In jump-like learning, a single run's mean accuracy mainly reports jump timing, and near-inert perturbations moved it by up to 0.28. Our preregistered design compared every arm against one base run per seed. Against a replicated reference, two B_ext contrasts change sign (Δ_K and S_{fade}), and the preregistered scaffold collapse shrinks from -0.27 to -0.10, with an interval that includes zero. Inert-perturbation replicates, such as our permuted-factor controls, are a cheap fix.

Third, compact causal mediators of a fine-tuned skill can be output readouts. The four-direction structure that passed ablation and transplant tests is the letter readout. Interchange tests across answer formats are a cheap safeguard.

Fourth, a large kernel change is not a useful process change by default. A's kernel differs by orders of magnitude, but it is not aligned with B, and from a matched start it does not speed learning on average. Post hoc, the one statistic that tracked which adapters helped measured alignment under the optimizer's geometry, not scale. That is a hypothesis, frozen for a held-out test.

9 Limitations

- One small model (Qwen3-1.7B) and a templated world; transfer is partly lexical. The preregistered 0.6B replication was not run.
- A is learned from outcome labels weighted by the model's policy, not from sampled choices.
- The corrected analysis is post hoc. It treats the perm controls as replicates of base; they are minimal perturbations, and independent replicates (fresh ϕ initialisation or item order) might vary more.
- Six adapters and three B seeds. With three seeds, any resampling of the seed factor is coarse, and the intervals in Table 2 may still be too narrow.
- On B_ind the budget ends at the onset of learning, so B_ind speaks only to early learning. The budget rule was frozen before the noise was known.
- The output-matched learner inherits A's saturated softmax geometry. Saturation alone slows matched-start learning, which could mask a rule-specific benefit; the paired controls are underpowered.
- The prior-shift arm is degenerate under saturated offsets, so Δ_{start} is a frozen start and the interaction partly reflects the arm's inability to move.
- The preregistered sensitivity control failed. The learning-rate benchmark was added after the results and used one run per seed.
- One optimizer (Adam). Of our kernel statistics, only F_{sign} reflects its preconditioning, and results may differ under SGD.
- The matched learner is not purely kernel-only. The letter-mass term is A's, gradient clipping binds at different rates, and the fade arm's offsets were matched only on a 0.1 grid. The perm replicates are near-inert, not identical (0.6–1.5 nats mean letter log-probability difference).
- The adapter-level kernel result (E4) is post hoc, with six adapters, four candidate statistics, and kernels measured on one B seed.
- The audit trail has gaps. Hashes are self-hosted, without a public timestamp. D1 followed a smoke test, though it preceded all Stage-2 results. Four scripts were extended for Stage 3 after the Stage-2 freeze, and their Stage-2 versions were not archived. The script that implements the update (`h2lib.py`) is unchanged since the Stage-2 freeze.

- The output-matched null equalises predictions only at $t = 0$; it is a first-order construction. Analysis is at the adapter level, because Stage 0 found no single-site representation of A to remove.

10 Conclusion

We asked whether an experience-acquired abstraction changes how a later abstraction is learned, and built a decomposition that can tell in principle. In Qwen3-1.7B we find no evidence of a mean benefit. From matched starting predictions, any average benefit is below $+0.02$ in mean balanced accuracy at the onset of learning and below $+0.09$ later. Process effects are not ruled out either. Post hoc, saturated rule-free adapters slowed matched-start learning, though not significantly at the adapter level, and one kernel statistic ranked which A adapters help and which hurt; we have frozen that ranking as a forecast for held-out adapters. The acquired skill gives a head start, and a network effect that works only from that start; whether that effect is specific to the abstraction is unresolved. On the task whose rule A does not share, we see no scaffold effect at the onset of learning, the only stage our budget reached; on the task A extends, the question is open. The largest lesson is methodological. In this regime one training run is not a reference, and our own preregistered design overstated two conclusions because of it. Section 7 lists the design that could settle the question; much of it is preregistered.

Statements

Preregistration. Stage 1 was frozen on 2026-10-16 at 12:35 and Stage 2 at 23:32 (+04:00), each by sha256 together with the code. Stage 3 was frozen on 2026-10-17 at 09:28 and stopped during its pilot (D6). All deviations (D1–D6) and exploratory analyses (E1–E4) are listed in Appendix A. D1–D6, E1 and E2 were logged before their results were read. E3 and E4 were written after all Stage-2 results were known and are labelled post hoc throughout. All three stages were frozen within 21 hours, by hashes we host ourselves, and their design decisions were delegated to the AI system. Readers should treat the label as a record of intent and timing, not as an external guarantee. The F_{sign} forecast (Appendix I) is registered as a separate Zenodo record (doi:10.5281/zenodo.23432798), published before this preprint.

Code and data. The task generator, the training and analysis code, the preregistrations, the numeric results and this paper's source are at `https://github.com/Somebodyhere101/head-start-or-process`. Its release v1.0 carries every learned B adapter (`b-adapters-part01-of-09.zip` to `part09`) and the A and control adapters. This preprint's Zenodo record (doi:10.5281/zenodo.23432800) archives the code, the A and control adapters, the logs and per-run results (`code-and-data.zip`), the preregistrations and deviation log (`preregistrations.zip`), and the LaTeX source.

Use of AI assistance. Substantial AI assistance was used throughout this project. The question of whether an experience-acquired abstraction changes the process by which a later abstraction is formed, rather than only supplying information, is the author's. An AI system (Claude, Anthropic), working under the author's direction, audited the earlier studies, reviewed the literature, designed the study, wrote the preregistrations and code, ran the experiments and analyses, and drafted this paper. A separate instance of the same system reviewed the draft; that review led to the post hoc analyses E3 and E4. The preregistrations record that the author delegated the design decisions to the AI system. The author directed the work and is responsible for its content.

References

S. M. R. Arnold, S. Iqbal, and F. Sha. When MAML can adapt fast and how to assist when it cannot. *AISTATS*, 2021. arXiv:1910.13603.

J. T. Ash and R. P. Adams. On warm-starting neural network training. *NeurIPS*, 2020. arXiv:1910.08475.

X. Bouthillier et al. Accounting for variance in machine learning benchmarks. *MLSys*, 2021. arXiv:2103.03098.

A. C. Cameron, J. B. Gelbach, and D. L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008.

H. Casademunt, C. Juang, A. Karvonen, S. Marks, S. Rajamanoharan, and N. Nanda. Steering out-of-distribution generalization with concept ablation fine-tuning. arXiv:2507.16795, 2025.

A. Chen, R. Shwartz-Ziv, K. Cho, M. L. Leavitt, and N. Saphra. Sudden drops in the loss: Syntax acquisition, phase transitions, and simplicity bias in MLMs. *ICLR*, 2024. arXiv:2309.07311.

R. Chen, A. Arditi, H. Sleight, O. Evans, and J. Lindsey. Persona vectors: Monitoring and controlling character traits in language models. arXiv:2507.21509, 2025.

C. Clark, M. Yatskar, and L. Zettlemoyer. Don't take the easy way out: Ensemble based methods for avoiding known dataset biases. *EMNLP*, 2019. arXiv:1909.03683.

Y. Deng, J. Liu, K. Tan, and Y. Liu. Early learning shapes later directions of representation change in continual learning. arXiv:2609.36081, 2026.

A. Deshpande, A. Achille, A. Ravichandran, H. Li, L. Zancato, C. Fowlkes, R. Bhotika, S. Soatto, and P. Perona. A linearized framework and a new benchmark for model selection for fine-tuning. arXiv:2102.00084, 2021.

J. Dodge, G. Ilharco, R. Schwartz, A. Farhadi, H. Hajishirzi, and N. A. Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. arXiv:2002.06305, 2020.

S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.

C. Finn and S. Levine. Meta-learning and universality: Deep representations and gradient descent can approximate any learning algorithm. *ICLR*, 2018. arXiv:1710.11622.

A. Geiger, Z. Wu, C. Potts, T. Icard, and N. D. Goodman. Finding alignments between interpretable causal variables and distributed neural representations. *CLeaR*, 2024. arXiv:2303.02536.

V. Goudar, B. Peysakhovich, D. J. Freedman, E. A. Buffalo, and X.-J. Wang. Schema formation in a neural population subspace underlies learning-to-learn in flexible sensorimotor problem-solving. *Nature Neuroscience*, 26(5):879–890, 2023.

J. Guan, Y. Yang, Z. Liu, Y. Zhang, F. Meng, and J. Feng. Active adaptation, not static defense: Temporal dynamics of preventative steering in adversarial fine-tuning. arXiv:2609.10142, 2026.

H. F. Harlow. The formation of learning sets. *Psychological Review*, 56(1):51–65, 1949.

A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *NeurIPS*, 2018. arXiv:1806.07572.

S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. *ICML*, 2019. arXiv:1905.00414.

D. Krasheninnikov, E. Krasheninnikov, B. Mlodozeniec, T. Maharaj, and D. Krueger. Implicit meta-learning may lead language models to trust more reliable sources. *ICML*, PMLR 235:25534–25559, 2024.

A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. *ICLR*, 2022. arXiv:2202.10054.

T. Lieberum, M. Rahtz, J. Kramár, N. Nanda, G. Irving, R. Shah, and V. Mikulik. Does circuit analysis interpretability scale? Evidence from multiple choice capabilities in Chinchilla. arXiv:2307.09458, 2023.

C. Lyle, Z. Zheng, E. Nikishin, B. Avila Pires, R. Pascanu, and W. Dabney. Understanding plasticity in neural networks. *ICML*, 2023. arXiv:2303.01486.

A. Makelov, G. Lange, A. Geiger, and N. Nanda. Is this the subspace you are looking for? An interpretability illusion for subspace activation patching. *ICLR*, 2024. arXiv:2311.17030.

S. Malladi, A. Wettig, D. Yu, D. Chen, and S. Arora. A kernel-based view of language model fine-tuning. *ICML*, 2023. arXiv:2210.05643.

I. Maor et al. Persistent representation of a prior schema in the orbitofrontal cortex facilitates learning of a conflicting schema. *Nature Communications*, 17, 2026. doi:10.1038/s41467-026-69330-2.

J. Mei, C. Xiao, C. Szepesvári, and D. Schuurmans. On the global convergence rates of softmax policy gradient methods. *ICML*, 2020. arXiv:2005.06392.

M. Mosbach, M. Andriushchenko, and D. Klakow. On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. *ICLR*, 2021. arXiv:2006.04884.

G. Ortiz-Jiménez, S.-M. Moosavi-Dezfooli, and P. Frossard. What can linearized neural networks actually say about generalization? *NeurIPS*, 2021. arXiv:2106.06770.

A. B. Owen. The pigeonhole bootstrap. *The Annals of Applied Statistics*, 1(2):386–411, 2007.

N. Prakash, T. Rott Shaham, T. Haklay, Y. Belinkov, and D. Bau. Fine-tuning enhances existing mechanisms: A case study on entity tracking. *ICLR*, 2024. arXiv:2402.14811.

A. Raghu, M. Raghu, S. Bengio, and O. Vinyals. Rapid learning or feature reuse? Towards understanding the effectiveness of MAML. *ICLR*, 2020. arXiv:1909.09157.

Y. Ren and D. J. Sutherland. Learning dynamics of LLM finetuning. *ICLR*, 2025. arXiv:2407.10490.

A. K. Singh, T. Moskovitz, S. Dragutinović, F. Hill, S. C. Y. Chan, and A. M. Saxe. Strategy coopetition explains the emergence and transience of in-context learning. *ICML*, 2025. arXiv:2503.05631.

D. Tse, R. F. Langston, M. Kakeyama, I. Bethus, P. A. Spooner, E. R. Wood, M. P. Witter, and R. G. M. Morris. Schemas and memory consolidation. *Science*, 316(5821):76–82, 2007.

S. Wiegreffe, O. Tafjord, Y. Belinkov, H. Hajishirzi, and A. Sabharwal. Answer, assemble, ace: Understanding how LMs answer multiple choice questions. *ICLR*, 2025. arXiv:2407.15018.

Z. Yang, Z. Zhao, W. Tang, X. Jiang, Y. Liu, T. Chen, Z. Yu, and J. Liu. Training trajectories determine circuit removability in annealable soft-prior transformers. arXiv:2609.10287, 2026.

A Preregistration and Deviations

D1 The KL anchor uses a proper four-letter KL, with the mass term computed on the raw output. The original form overflows under output offsets. Found in a smoke test and applied to all arms before any Stage-2 result.

D2 Option-token sites were added to Stage 0, after a smoke test showed the final-position candidates were letter readouts.

D3 Flip and copy rates are measured as excess over unpatched pairs. An inert patch has a raw copy rate of 0.22 by chance.

D4 Adaptive light Stage 0 for adapters 2–6: whole-residual swaps bound any subspace swap.

D5 An analysis-script crash fixed: an unused required argument; no logic changed.

D6 Stage 3 was stopped by the author during its pilot. No Stage-3 result is reported.

E1, E2 Exploratory, logged before running: the learning-rate benchmark, the start $\times$ network factorial, and the saturation test (reading rule written before the run).

E3 Post hoc, written after all Stage-2 results were read: the four-run reference, the three-way bootstrap, t intervals and permutation tests, the decomposition of the E2 interaction, and the scaffold contrasts against the reference (Section 6.3).

E4 Post hoc: the preregistered forecast computed by a saved script, and the adapter-level ranking of Δ_K by $t = 0$ kernel statistics.

Pilot budgets were set by the frozen rule: the smallest number of passes at which base reached balanced accuracy ≥ 0.85 on one or two of three pilot seeds. This gave one pass for B_ind and three for B_ext. Output-matching exactness was $\leq 8 \times 10^{-6}$ in every cell.

B Training and Evaluation Details

- **Update (frozen recipe v1).** Exact policy gradient over the four letters with rewards ± 1 per letter; policy temperature $\tau = 2$; $\epsilon = 0.1$ uniform mixture on the policy weights; moving-average baseline (decay 0.9); mass term weight 0.1, on the raw letter output; KL anchor weight 0.03 (four-letter KL, D1); Adam, learning rate 5×10^{-4}; batch 4; gradient-norm clip 1.0.
- **Adapters.** Hand-written LoRA, rank 8, $\alpha = 16$ (scale 2), on the query and value projections of every layer. ϕ is initialised from the B seed and is identical across arms.
- B **data.** Per B seed: 192 training situations (fixed order), 96 held-out evaluation situations and 96 shifted-distribution situations, all generated from the B seed alone. G averages balanced accuracy at eight evenly spaced points over the budget (every 24 situations on B_ind, every 72 on B_ext), excluding $t = 0$.
- A **adapters.** Six designated adapters from three training series (ST2_20260919, ST2_20260921, ST2_20260922, ST2b_20260920, P1_20260923, P1_20260924); P1_20260925–28 are held out for confirmation.

- **Controls.** shuf, single and perm adapters were trained (or, for perm, constructed) for seeds 20260919, 20260921 and 20260922, the same seeds as the first three A adapters.

C Per-Adapter Results

	B_ind				B_ext			
A adapter	Δ_{raw}	Δ_{start}	Δ_K	S	Δ_{raw}	Δ_{start}	Δ_K	S
ST2_20260919	$+0.254$	$+0.068$	-0.036	-0.049	$+0.359$	$+0.200$	$+0.071$	-0.312
ST2_20260921	$+0.135$	$+0.057$	$+0.029$	$+0.010$	$+0.284$	$+0.181$	-0.059	-0.226
ST2_20260922	$+0.132$	$+0.105$	-0.089	-0.045	$+0.280$	$+0.148$	-0.120	-0.181
ST2b_20260920	$+0.099$	$+0.058$	$+0.015$	-0.052	$+0.147$	$+0.174$	-0.064	-0.299
P1_20260923	$+0.109$	$+0.074$	-0.047	-0.010	$+0.261$	$+0.155$	-0.082	-0.295
P1_20260924	$+0.207$	$+0.039$	$+0.046$	-0.066	$+0.371$	$+0.157$	$+0.082$	-0.323

Table 3: Preregistered contrasts per A adapter (against the single base run), averaged over B seeds 101–103 (Qwen3-1.7B).

D Reference Runs

		G				final accuracy			
Task	seed	base	perm 19	perm 21	perm 22	base	perm 19	perm 21	perm 22
B_ind	101	0.504	0.508	0.518	0.522	0.51	0.53	0.58	0.64
B_ind	102	0.526	0.526	0.540	0.539	0.55	0.54	0.68	0.62
B_ind	103	0.539	0.615	0.518	0.522	0.64	0.75	0.53	0.57
B_ext	101	0.706	0.422	0.566	0.620	0.99	0.42	0.86	0.50
B_ext	102	0.527	0.520	0.517	0.505	0.65	0.56	0.50	0.50
B_ext	103	0.586	0.542	0.521	0.652	0.83	0.50	0.58	0.84

Table 4: The four reference runs per B seed: base and the output-matched perm controls (near-inert perturbations of base). Within-seed SD of G: 0.027 (B_ind), 0.077 (B_ext); of final accuracy: 0.074 and 0.194.

E Control Adapters

Control	software accuracy	letter KL to base	Δ_K B_ind	Δ_K B_ext
shuf_20260919	0.531	11.36	-0.006	-0.114
shuf_20260921	0.469	7.59	-0.072	-0.136
shuf_20260922	0.469	10.08	-0.029	-0.047
single_20260919	0.750	5.75	-0.015	$+0.010$
single_20260921	0.750	5.75	$+0.137$	-0.108
single_20260922	0.750	5.75	$+0.083$	-0.106
perm_20260919	0.516	0.00	$+0.026$	-0.112
perm_20260921	0.516	0.01	$+0.003$	-0.072
perm_20260922	0.516	0.00	$+0.005$	-0.014

Table 5: Control adapters, trained with the same recipe as A. *Software accuracy* is meaning accuracy on fresh items under A's rule (base 0.516, every A adapter 1.000); a single-cue policy scores 0.75. *Letter KL* is KL(control $\|$ base) over the four letters on the same items (A adapter 1: 10.35). For a saturated adapter it measures disagreement with base, not saturation: the three single controls make identical choices and so have identical KL. Saturation, as the mean $|\delta|$: A 75–82 nats, shuf 61–75, single 74–77, perm 0.6–1.5. Δ_K is the output-matched learning advantage over the single base run.

F Stage 0 Detail

Each A adapter was tested on 32 fresh situations per domain $\times$ 4 option rotations (fit on plant and clinic, test on software). Base meaning accuracy was 0.504 (fit) and 0.516 (test); every adapter scored 1.000 on both.

Adapter 1 (ST2_20260919) ran the full candidate set at every site and layer: top-4 PCs of A's delta; letter-residualised PCs ($k = 2, 4, 16$); a class-mean decision direction; random subspaces ($k = 1, 4$); the base model's letter-identity subspace; and the whole residual. Adapters 2–6 ran the whole-residual swap at every site, plus the top-4 PCs at the final position. Their runs crashed in a final control step after every site had been logged. Their patching results are recovered from the logs, which record the fit domains only, so the software-domain confirmation exists for adapter 1 alone. A full candidate set would have been run wherever a whole-residual swap moved meaning by ≥ 0.2; this never happened.

Excess meaning-flip was 0.00 for every candidate, site and layer on all six adapters. At the final position, excess letter-copy was 0.00 at layer 16; 0.00–0.43 at layer 20 (varying by adapter); and 0.64 at layers 24 and 26, with the top-4 subspace identical to the whole residual.

G Tangent-Kernel Statistics at the Start of B

Task	Network	kernel scale	F_{res}	F_{sign}	item-type CKA
B_ind	base	1.31×10^4	74.6	82.6	0.46
B_ind	A (matched frame)	1.65×10^5–1.7×10^6	1.2–8.8	−3.2–41.7	0.01–0.05
B_ind	shuf / single (matched)	912–9.71×10^5	5.6–25.4	11.2–66.6	0.03–0.16
B_ind	perm (matched)	1.31×10^4–1.35×10^4	72.3–74.7	78.3–82.2	0.45–0.49
B_ext	base	1.06×10^4	84.5	90.1	0.53
B_ext	A (matched frame)	1.2×10^3–3.76×10^7	1.6–11.6	−30.8–58.4	0.01–0.08
B_ext	shuf / single (matched)	573–6.61×10^5	6.9–26.9	12.6–56.1	0.05–0.17
B_ext	perm (matched)	9.9×10^3–1.18×10^4	82.9–89.0	86.9–92.5	0.50–0.57

Table 6: Empirical tangent kernel of the fresh B adapter on B's 192 training prompts, measured before any update (B seed 101). The output is the log-probability of the correct letter in each arm's frame. Kernel scale is the mean diagonal of $JJ^\top$. $F_{\text{res}} = d^\top K d / (\|d\|^2 \operatorname{diag} K)$ with $d_i = 1 - q_i$, for the plain gradient kernel; F_{sign} is the same statistic for the sign-gradient kernel, which approximates Adam's update. CKA is computed against the item-type indicator kernel. Ranges are over adapters.

H Process-Only Control: Calibration

Amplification factor	mean G gain over base, B_ind	B_ext
×2	−0.059	+0.070
×4	−0.030	+0.079
×8	−0.048	+0.011

Table 7: The preregistered process-only control. Its forward pass is base's, and after each step the update's component in an 8-dimensional calibration-gradient subspace is amplified. The preregistered rule required ≥ 0.10 on both tasks; no factor qualified. Calibration used pilot seeds 1–3, each against a single base run.

I Frozen Forecast for the Held-Out Adapters

Four A adapters (P1_20260925–28) were designated as a held-out confirmation set in the Stage-1 preregistration and have never been used for B learning. Their far-domain transfer is 0.976–0.990, like the six used adapters. Before any B-learning run on them, we measured F_{sign} in the output-matched frame at $t = 0$ (B seed 101, as for the six; E5) and froze the forecast below in `FSIGN_FORECAST.md` (sha256 `0400cbfb03436aa5...`), registered as Zenodo record doi:10.5281/zenodo.23432798.

Held-out adapter	F_{sign}, B_ind	F_{sign}, B_ext	predicted rank, B_ind	predicted rank, B_ext
P1_20260925	34.0	46.6	1	1
P1_20260926	25.5	13.6	4	2
P1_20260927	26.6	9.3	3	3
P1_20260928	28.2	4.1	2	4

Table 8: The frozen forecast. Rank 1 is the largest predicted Δ_K. The test, the analysis and the replicated-reference design are specified in `FSIGN_FORECAST.md`.

J Environment

- Model: Qwen3-1.7B (Hugging Face snapshot `70d244cc`), bf16, with hand-written rank-8 LoRA on the query and value projections (scale 2).
- Hardware and software: one laptop RTX 5070 (8 GB); torch 2.7.1+cu128; transformers 5.16.1.
- Run times: a B-learning arm took 20–120 s, and the Stage-2 grid about 6 h of GPU time.
- Determinism: an identical configuration reproduced G exactly (replicate difference 0.000). Near-inert changes to the weights do not (Section 6.2).